

Team Number:	apmcm253XXXXX
Problem Chosen:	A

2025 APMCM summary sheet

apmcmthesis $\text{\LaTeX}$ template is designed by <https://www.latexstudio.net> for <http://www.apmcm.org>. The template is designed to let everyone focus on the content writing of the paper, without spending too much effort on the customization and adjustment of the format.

Note that users need to have some experience with $\text{\LaTeX}$, at least some of the features of common macro packages, such as references, math formulas, image usage, list environment, etc. Templates have added commonly used macros. Package, no additional user added.

This template is located on <https://github.com/latexstudio/APMCMThesis>. You can update files from the repository.

Keywords: Keywords1 Keywords2 Keywords3

Contents

1. Introduction.....	1
1.1	1
1.2	1
1.3	1
2. The Description of the Problem.....	1
2.1 How do we approximate the whole course of ?.....	1
2.2 How do we define the optimal configuration?.....	1
2.3 The local optimization and the overall optimization.....	1
2.4 The differences in weights and sizes of.....	2
2.5 What if there is no data available?	2
3. Models	2
3.1 Basic Model.....	2
3.1.1 <i>Terms, Definitions and Symbols</i>	2
3.1.2 <i>Assumptions</i>	2
3.1.3 <i>The Foundation of Model</i>	2
3.1.4 <i>Solution and Result</i>	2
3.1.5 <i>Analysis of the Result</i>	2
3.1.6 <i>Strength and Weakness</i>	3
4. Conclusions	3
4.1 Conclusions of the problem.....	3
4.2 Methods used in our models	3
4.3 Applications of our models.....	3
5. Future Work	4
5.1 Another model.....	4
5.1.1 <i>The limitations of queuing theory</i>	4
5.1.2	4
5.1.3	4
5.1.4	4
6. References	4
7. Appendix	5

I. Introduction

In order to indicate the origin of problems, the following background is worth mentioning.

1.1

1.2

1.3

II. The Description of the Problem

2.1 How do we approximate the whole course of ?

-
-
-

2.2 How do we define the optimal configuration?

- 1) From the perspective of :
- 2) From the perspective of the :
- 3) Compromise:

2.3 The local optimization and the overall optimization

-
-
- Virtually:

2.4 The differences in weights and sizes of

2.5 What if there is no data available?

III. Models

3.1 Basic Model

3.1.1 *Terms, Definitions and Symbols*

The signs and definitions are mostly generated from queuing theory.

3.1.2 *Assumptions*

3.1.3 *The Foundation of Model*

1) The utility function

- The cost of :
- The loss of :
- The weight of each aspect:
- Compromise:

3) The overall optimization and the local optimization

- The overall optimization:
- The local optimization:
- The optimal number of :

3.1.4 *Solution and Result*

1) The solution of the integer programming: 2) Results:

3.1.5 *Analysis of the Result*

- Local optimization and overall optimization:
- Sensitivity: The result is quite sensitive to the change of the three parameters
-
- Trend:
- Comparison:

3.1.6 *Strength and Weakness*

Strength: The Improved Model aims to make up for the neglect of . The result seems to declare that this model is more reasonable than the Basic Model and much more effective than the existing design.

Weakness: Thus the model is still an approximate on a large scale. This has doomed to limit the applications of it.

IV. Conclusions

4.1 Conclusions of the problem

-
-
-
-

4.2 Methods used in our models

-
-
-
-

4.3 Applications of our models

-
-
-
-

V. Future Work

5.1 Another model

5.1.1 *The limitations of queuing theory*

5.1.2

5.1.3

5.1.4

VI. References

[1] Author, Title, Place of Publication: Press, Year of publication.

VII. Appendix

Listing 1: The matlab Source code of Algorithm

```

kk=2; [mdd, ndd]=size(dd);
while ~isempty(V)
    [tmpd, j]=min(W(i, V)); tmpj=V(j);
    for k=2:ndd
        [tmp1, jj]=min(dd(1, k)+W(dd(2, k), V));
        tmp2=V(jj); tt(k-1, :)= [tmp1, tmp2, jj];
    end
    tmp=[tmpd, tmpj, j; tt]; [tmp3, tmp4]=min(tmp(:, 1));
    if tmp3==tmpd, ss(1:2, kk)=[i; tmp(tmp4, 2)];
    else, tmp5=find(ss(:, tmp4)~=0); tmp6=length(tmp5);
    if dd(2, tmp4)==ss(tmp6, tmp4)
        ss(1:tmp6+1, kk)=[ss(tmp5, tmp4); tmp(tmp4, 2)];
    else, ss(1:3, kk)=[i; dd(2, tmp4); tmp(tmp4, 2)];
    end; end
    dd=[dd, [tmp3; tmp(tmp4, 2)]]; V(tmp(tmp4, 3))=[];
    [mdd, ndd]=size(dd); kk=kk+1;
end; S=ss; D=dd(1, :);

```

Listing 2: The lingo source code

```

kk=2;
[mdd, ndd]=size(dd);
while ~isempty(V)
    [tmpd, j]=min(W(i, V)); tmpj=V(j);
    for k=2:ndd
        [tmp1, jj]=min(dd(1, k)+W(dd(2, k), V));
        tmp2=V(jj); tt(k-1, :)= [tmp1, tmp2, jj];
    end
    tmp=[tmpd, tmpj, j; tt]; [tmp3, tmp4]=min(tmp(:, 1));
    if tmp3==tmpd, ss(1:2, kk)=[i; tmp(tmp4, 2)];
    else, tmp5=find(ss(:, tmp4)~=0); tmp6=length(tmp5);
    if dd(2, tmp4)==ss(tmp6, tmp4)
        ss(1:tmp6+1, kk)=[ss(tmp5, tmp4); tmp(tmp4, 2)];
    else, ss(1:3, kk)=[i; dd(2, tmp4); tmp(tmp4, 2)];

```

```
end;  
end  
    dd=[dd,[tmp3;tmp(tmp4,2)]];V(tmp(tmp4,3))=[];  
    [mdd,ndd]=size(dd);  
    kk=kk+1;  
end;  
S=ss;  
D=dd(1,:);
```